实验5：LLM、Context 与 RAG 实现

项目名称： AI象棋对弈助手系统

班级： 24级软件工程1班

学号： 24111302009

姓名： 闫俊

知识库与代码版本： V1.0

本次云端模型： DeepSeek `deepseek-chat`，接口返回成功，耗时约 1181.55 ms

一、实验目标与实现范围

本实验在实验4的本地象棋练习程序基础上，增加一个面向项目知识问答的 LLM、Context 与 RAG 最小链路。问题任务对应实验2的 US-03（查看单步解释）和 US-04（对局后复盘摘要）：用户询问当前版本的规则、保存方式和 AI 能力边界，系统检索项目资料后生成带来源编号的回答。

本版本提供两种模型模式：`offline` 使用可重复的本地演示模型，用于验证切分、检索、上下文和引用；`cloud` 使用 OpenAI Chat Completions 兼容接口，读取 `LLM_API_KEY`、`LLM_MODEL` 和 `LLM_ENDPOINT` 环境变量，不把密钥写入代码、日志或仓库。本次实际云端测试使用 DeepSeek `deepseek-chat`，返回了带 `K-002` 来源的回答；截图见图7。API Key 仅在本地环境变量中使用，没有写入代码、证据 JSON 或仓库。

二、代码、入口与运行方式

路径	作用
`app/exp5_rag.py`	文档清理、切分、关键词检索、Prompt、离线模型、云端兼容调用和对比测试
`docs/knowledge/xiangqi_rules.md`	资料来源明确的项目知识库
`tests/test_exp5_rag.py`	切分、检索、引用、资料不足和对比测试的自动化检查
`docs/evidence/exp5/`	每次运行产生的 JSON 证据文件

在项目根目录运行：

```
.\.venv\Scripts\python.exe -m app.exp5_rag --question "        " --mode offline
.\.venv\Scripts\python.exe -m app.exp5_rag --suite --mode offline
.\.venv\Scripts\python.exe -m app.exp5_rag --failure-test
.\.venv\Scripts\python.exe -m pytest tests\test_exp5_rag.py -v
```

云端模式（只有你拥有可用 API Key 时执行）：

```
$env:LLM_API_KEY = "  API  "
$env:LLM_MODEL = "gpt-4o-mini"
$env:LLM_ENDPOINT = "https://api.openai.com/v1"
.\.venv\Scripts\python.exe -m app.exp5_rag --question "            " --mode cloud
```

不要截图或提交包含 API Key 的窗口；运行结束后可执行 `Remove-Item Env:LLM_API_KEY` 清除当前 PowerShell 会话变量。

三、知识库处理与切分策略

知识库只有一份项目自有 Markdown 资料，内容来自实验4已实现功能和当前限制，不包含姓名、密码、Token 或其他敏感信息。资料格式为 UTF-8 Markdown，数量为1份；更新方式是修改原始 Markdown 后重新运行程序，程序启动时重新读取并切分，不使用旧索引。

切分规则：先按标题和段落分组；超过 360 字符时按句号或换行切分，窗口最大 360 字符，相邻窗口重叠 60 字符。每个片段带 `chunk_id`、文件名和标题元数据。当前知识库很小，因此关键词检索比引入向量数据库更容易解释、复现和审查；后续资料规模扩大后再考虑同一嵌入模型的向量检索。

抽查片段时重点检查三项：关键语句没有被截断、来源文件名与原资料一致、片段标题可以帮助定位上下文。运行结果中的 `chunks` 字段保存 Top-k 片段及匹配词，便于复核。

四、Prompt 与上下文工程

原版 Prompt `PROMPT_V1` 只给出角色和用户问题，缺少资料边界、输出格式和来源要求。修订版 `PROMPT_V2` 增加四项约束：只依据检索资料、资料不足时明确提示、先结论后要点、事实后标记 [来源: K-xxx]。因此可以比较 Prompt 长度和回答变化，且每次查询使用独立上下文。

实际发送的完整输入保存在 `docs/evidence/exp5/query-offline-v2.json` 的 `prompt` 字段；其中同时保存问题、Top-k 片段、来源编号、回答和耗时。输入检查会删除重复的空白和无关文件，不把实验4的全部代码、日志或个人信息放入上下文。

五、检索与 RAG 最小链路

关键词检索把问题和片段分别切成中文词串、英文词和数字，按词语出现次数计算分数，再按分数降序取 Top-k，默认 `k=3`。返回片段仅表示词语相关，不等于事实一定正确；因此报告要求人工核对片段是否支持回答，并检查来源编号是否来自返回结果。

完整链路为：问题输入 → 关键词检索 → Top-k 排序 → 组织带来源上下文 → 模型调用 → 带出处回答。无匹配片段时返回“资料不足”，不会强行添加引用。`run_query` 的 JSON 结果保存检索片段、实际 Prompt、最终回答、错误信息和耗时。

六、固定问题与对比测试

固定问题集共6题，覆盖单片段、多片段、资料无答案和含义不清场景：

编号	问题类型	判断规则
Q1	单片段：棋局保存	必须说出 `data/games.json`，引用对应片段
Q2	单片段：兵卒规则	必须区分过河前后，引用规则片段
Q3	多片段综合：RAG链路	至少说明检索、上下文、来源和回答四步
Q4	能力边界：真实引擎	必须说明当前尚未接入，不得写成已完成
Q5	资料无答案：GitHub密码	应提示资料不足，不能编造操作步骤
Q6	含义不清：绝对最优	应说明当前版本不能保证，指出规则/引擎限制

直接模型组使用 `PROMPT_V1`，不提供检索资料；RAG 组使用 `PROMPT_V2`，提供同一知识库的 Top-k 资料和来源要求。每题重新建立输入，不把前题回答放入后题上下文。运行结果保存在 `comparison-offline.json`；云端模式结果需在设置 API Key 后单独生成。

统一记录回答正确性、要点遗漏、无依据内容、引用可靠性和耗时。离线演示模型的耗时仅用于验证程序链路，不能当作云端模型延迟或费用；云端费用和 Token 统计只有接口返回 usage 或账单数据时填写。

七、模块失败处理与 AI 辅助审查

`make_model` 在云端模式缺少 `LLM_API_KEY` 时返回明确错误；网络错误、超时、JSON 格式异常和返回字段缺失统一转换为“模型调用失败”，不让程序崩溃。`--failure-test` 用于保存一次连接失败或缺少 Key 证据。

本次代码审查重点是：是否把 API Key 写入仓库、是否无资料仍然生成引用、是否把检索相关误写成事实正确、是否混淆离线演示模型和云端模型。人工测试使用自动化用例、固定问题和 JSON 证据交叉检查；代码没有把离线演示标记为真实模型。

八、截图与操作记录

本次截图均来自项目根目录的 PyCharm Terminal，截图中没有显示 API Key。每张图对应一次实际运行或测试：

图1：离线 RAG 单次查询。 问题为“棋局保存在哪里？”，检索命中 `K-003`，回答给出 `data/games.json` 和来源编号。



图1 离线 RAG 单次查询

图2：Prompt V1。 原版 Prompt

只提供角色和用户问题，返回基本回答和来源信息，但没有资料边界、回答格式和不足处理要求。



图2 Prompt V1 运行结果

图3：Prompt V2。 修订版加入只依据资料、资料不足提示、结论加要点和来源标注要求；同一问题仍命中 `K-003`，输出约束更清晰。



图3 Prompt V2 运行结果

图4：固定问题对比测试。 共6个独立问题，直接模型组不传资料，RAG 组返回相关来源；资料无答案的 Q5、Q6 显示 `sources=none`。



图4 直接模型与 RAG 对比

图5：失败处理。 未设置 `LLM_API_KEY` 时，云端模式返回“缺少 LLM_API_KEY”错误并保存 `failure-cloud.json`，程序没有崩溃。



图5 云端调用失败处理

图6：自动化测试。 tests/test_exp5_rag.py
共6项测试全部通过，验证知识库切分、检索排序、来源引用、资料不足、Prompt 修订和对比测试结构。

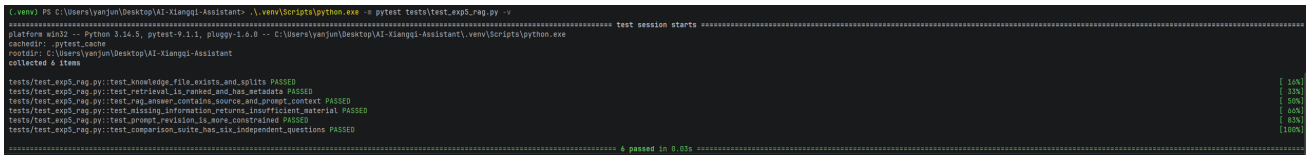


图6 实验5自动化测试

图7：真实云端调用。 使用 DeepSeek deepseek-chat 成功返回回答，检索命中 K-002，耗时约 1181.55 ms，并生成 query-cloud-v2.json。截图上方只显示模型和接口配置，不包含 API Key。



图7 DeepSeek 云端 RAG 调用

对应证据文件包括
query-offline-v1.json、query-offline-v2.json、query-cloud-v2.json、comparison-offline.json 和 failure-cloud.json；控制台文字记录保存在 docs/evidence/exp5/。

九、个人软件中的后续使用与局限

RAG 适合继续用于本项目的规则说明、功能帮助、复盘术语解释和版本限制提示；它不负责判定合法走法、将军/将死或生成绝对最优着法，这些仍由确定性规则模块和经过验证的象棋引擎负责。后续计划是扩充来源明确的规则资料、加入向量检索评估、增加引用可靠性测试，并把问答入口嵌入对弈和复盘页面。

当前局限包括：知识库只有一份短文档；关键词检索不能理解同义表达；离线演示模型不是云端 LLM；没有真实 Token/费用统计；尚未把 RAG 入口接入 PySide6
主界面。因此本实验结论只适用于本知识库、固定问题集和本次运行环境。

十、实验小结

本实验完成了资料清理、切分、关键词检索、来源编号、上下文组织、模型调用接口、失败处理和直接模型/RAG 对比测试的最小闭环。通过 Prompt V1 到 V2 的修订，回答边界和引用要求更加明确；通过固定问题集，可以分别观察资料充分、资料不足和能力边界问题。真实云端效果、Token/费用和最终截图必须以实际 API 调用记录为准。

十一、仓库证据

实验5文件已整理到：docs/experiments/exp5-llm-context-rag-v1.md、docs/knowledge/、app/exp5_rag.py、tests/test_exp5_rag.py 和 docs/evidence/exp5/。本报告 PDF 与实验5代码使用同一提交记录，仓库链接为 https://github.com/jyan5668-hub/AI-Xiangqi-Assistant。